

MID を使用した解約率予測 GLM の改良 - EV 計算編

Reiji Kozuma^{*} Ryoichi Asashiba[†] Yoshihiro Matsumori[‡] Hirokazu Iwasawa[§]

2026-09-25

概要

本ドキュメントは、元論文 Kozuma et al. (2026) にかかる計算のうち、EV の計算について具体的なコードとともに説明を補足するものである。なお、元論文に述べた内容は議論の前提とする。

目次

1 準備

- 1.1 パッケージとデータの準備
- 1.2 XGBoost モデルの構築

2 EV 計算の実装

- 2.1 群団の分割
- 2.2 将来契約ポートフォリオ作成用補助関数
- 2.3 将来保険料収入の設定
- 2.4 将来 CF 展開・利益現価計算

3 EV 変動要因分析の計算

Acknowledgments

References

^{*} Daido Life Insurance Company, rktkdm@gmail.com

[†] Daido Life Insurance Company, ryoichi.asashiba@gmail.com

[‡] Sompo Japan Insurance Inc., ymatsumori1@sompo-japan.co.jp

[§] Graduate School of Accountancy, Waseda University, iwahiro@bb.mbn.or.jp

1 準備

1.1 パッケージとデータの準備

本ドキュメントで使用するパッケージを読み込む。

```
library(midr)
library(tidyverse)
library(readxl)
library(xgboost)
library(waterfalls)
```

データの読み込みとその前処理は基本編と同一であるため、本ドキュメントでは記載を省略する。
データフレームから必要な列のみを取り出す部分を関数化する。

```
# Convert to model matrix
get_modelmatrix <- function(df){
  df <- prep_toint_pjratio(df)
  df <- model.matrix(~ ., df) # An intercept column of all 1s is included
}
get_xo <- function(df){ # All predictors and target variables
  df %>% select(setdiff(names(df),
                        c("ID", "lapse_study_year", "policy_year", "issue_year",
                          ↪ "exposure_amount")))
}
get_x <- function(df){ # Predictors only
  df <- df %>% get_xo
  df <- df %>% select(setdiff(names(df), c("lapse", "exposure", "lapse_rate")))
}
```

1.2 XGBoost モデルの構築

基本編と同様の手順で、XGBoost の予測モデルを構築する。ハイパーパラメータも同一である。本ドキュメントでは評価年度（Evaluation Year）2002 年度末から 2003 年度末への EV 変動要因分析を題材とするため、それぞれの時点で予測モデルを構築する必要がある。

```
year_start <- 2002
year_end <- 2003
df_train_start <- df_all %>%
  filter(lapse_study_year >= year_start-2, lapse_study_year <= year_start)
df_test_start <- df_all %>%
  filter(lapse_study_year == year_start+1)
df_train_end <- df_all %>%
  filter(lapse_study_year >= year_end-2, lapse_study_year <= year_end)
df_test_end <- df_all %>%
  filter(lapse_study_year == year_end+1)
```

```
get_mm <- function(df){
```

```

df <- df %>%
  select(setdiff(names(df), c(
    "ID", "lapse_study_year", "policy_year", "issue_year",
    "exposure_amount", "lapse", "exposure", "lapse_rate"))) %>%
  prep_toint_pjratio
model.matrix(~ ., df)
}

df_train_start_mm <- df_train_start %>% get_mm
constant_cols_start <- which(apply(df_train_start_mm, 2, function(col) all(col == col[1])))
dm_train_start <- xgb.DMatrix(
  data = df_train_start_mm[, -constant_cols_start, drop = FALSE],
  label = df_train_start %>% pull(lapse_rate),
  weight = df_train_start %>% pull(exposure))
df_train_end_mm <- df_train_end %>% get_mm
constant_cols_end <- which(apply(df_train_end_mm, 2, function(col) all(col == col[1])))
dm_train_end <- xgb.DMatrix(
  data = df_train_end_mm[, -constant_cols_end, drop = FALSE],
  label = df_train_end %>% pull(lapse_rate),
  weight = df_train_end %>% pull(exposure))

```

```

set.seed(2025)
cv_xgboost_start <- xgb.cv(data = dm_train_start, nrounds = 2000, nfold = 4,
  early_stopping_rounds = 10, params = params_xgb, verbose = 0)
nrounds <- cv_xgboost_start$early_stop$best_iteration

set.seed(2025)
model_xgboost_start_raw <- xgb.train(data = dm_train_start, nrounds = nrounds,
  params = params_xgb)

set.seed(2025)
cv_xgboost_end <- xgb.cv(data = dm_train_end, nrounds = 2000, nfold = 4,
  early_stopping_rounds = 10, params = params_xgb, verbose = 0)
nrounds <- cv_xgboost_end$early_stop$best_iteration

set.seed(2025)
model_xgboost_end_raw <- xgb.train(data = dm_train_end, nrounds = nrounds,
  params = params_xgb)

```

後のコードの見通しを良くするため、予測モデルの予測値の計算に必要な処理をまとめてひとつの関数にしておく。学習データですべての行で同じ数値となっている列がある場合は、新たなデータを予測モデルに投入する場合もその列のデータを取り除く必要がある。この部分の処理も前述の関数に含めたうえで、必要な情報をモデルとともに同じリストに格納しておくこととする。

```

model_xgboost_start <- list(
  model = model_xgboost_start_raw,
  constant_cols = constant_cols_start
)
model_xgboost_end <- list(
  model = model_xgboost_end_raw,

```

```

constant_cols = constant_cols_end
)

predfun_xgboost <- function(model, newdata){
  tmp <- newdata %>% get_x %>% get_modelmatrix
  tmp <- tmp[, -model$constant_cols, drop = FALSE]
  predict(model$model, tmp)
}

```

2 EV 計算の実装

2.1 群団の分割

将来プロジェクトにあたっては説明変数 `duration` を 1 年ごとに順次加算して解約率を予測する。しかし、実際のデータでは `duration` はいくつかにグルーピングされているため、このグルーピングを解除し、1 年単位のレコードに分割する必要がある。

```

df_all %>% group_by(lapse_study_year, duration, issue_year) %>%
  summarise(sum(exposure), .groups = "drop")

```

```

# A tibble: 91 x 4
  lapse_study_year duration issue_year `sum(exposure)`
      <int> <ord>      <ord>          <dbl>
1         2000 06-09   1991-1993      36406.
2         2000 06-09   1994-1996      60465.
3         2000 10      1991-1993      14055.
4         2000 11      -1990         3746.
5         2000 12      -1990          348.
6         2000 13-     -1990          185
7         2001 06-09   1991-1993      23212.
8         2001 06-09   1994-1996     113966.
9         2001 10      1991-1993      18372.
10        2001 11      1991-1993      10989.

```

```
# i 81 more rows
```

ここで契約年 `issue_year` の情報を活用すれば、ありえる `duration` の範囲を絞り込むことができる。

たとえば、`lapse_study_year == 2000 & duration == 6-9 & issue_year == 1991-1993` の組み合わせのレコードに対しては、`lapse_study_year = issue_year + duration - 1` の関係式から `duration` は 8 以上 10 以下となる。しかし 10 は `duration == 6-9` の条件に反する。したがって、残る可能性は 8, 9 となる。

これ以上 `duration` の範囲を絞り込むことは出来ないで、8 のレコードと 9 のレコードに 2 等分に分割する。同じように、複数の `duration` がありえるキーの組み合わせに対しては、等分することで `duration` のグルーピングを解除する。

加えて、`issue_year` には Pre-1991 という群団があるが、実際の `issue_year` が推定できる `lapse_study_year == 2000` のレコードを確かめてみると `duration == 13-` (実際の契約年は 1988 年以

前) のレコードが他に比べて非常に少ないことが分かる。

```
df_all %>% filter(issue_year == "-1990", lapse_study_year < 2004) %>%
  group_by(lapse_study_year, duration, issue_year) %>%
  summarise(sum(exposure), .groups = "drop")
```

A tibble: 7 x 4

	lapse_study_year	duration	issue_year	`sum(exposure)`
	<int>	<ord>	<ord>	<dbl>
1	2000	11	-1990	3746.
2	2000	12	-1990	348.
3	2000	13-	-1990	185
4	2001	12	-1990	2766.
5	2001	13-	-1990	435.
6	2002	13-	-1990	2830.
7	2003	13-	-1990	2590.

そのため、1987 年以前契約のボリュームはほとんどないと判断し、issue_year は 1988 年以降であると仮定する。

このような方法でグルーピングを解除した duration を今しばらく duration_estimated という名前で区別することとする。また、issue_year も lapse_study_year = issue_year + duration - 1 の関係式から判定できる。duration_estimated にこの関係式を当てはめて判定したものを、同じように issue_year_estimated とする。

実際の分割処理は、もともとのキーの組み合わせに対して、分割すべきキーとレコードを分割するための係数を対応付けるテーブルを用意し、これと left_join() 関数により結合することで行う。

```
df_conv_issue_year_estimated <- data.frame(
  issue_year = c("-1990", "-1990", "-1990",
                 "1991-1993", "1991-1993", "1991-1993",
                 "1994-1996", "1994-1996", "1994-1996",
                 "1997-1999", "1997-1999", "1997-1999",
                 "2000-2002", "2000-2002", "2000-2002",
                 "2003-2004", "2003-2004",
                 "2005-2007", "2005-2007", "2005-2007") %>%
  factor(levels = levels(df_all$issue_year), ordered = TRUE),
  issue_year_estimated = 1988:2007
)

df_conv_duration_estimated <- data.frame(
  duration = c("06-09", "06-09", "06-09", "06-09",
              "10", "11", "12",
              rep("13-", 55 - 13 + 1)) %>%
  factor(levels = levels(df_all$duration), ordered = TRUE),
  duration_estimated = 6:55
)
```

```

# Enumerate all valid combinations
df_conv_estimated <- expand.grid(
  lapse_study_year = unique(df_all$lapse_study_year),
  duration = unique(df_all$duration),
  issue_year = unique(df_all$issue_year)
) %>%
left_join(df_conv_issue_year_estimated, join_by(issue_year == issue_year), relationship =
  ↪ "many-to-many") %>%
left_join(df_conv_duration_estimated, join_by(duration == duration), relationship =
  ↪ "many-to-many") %>%
filter(issue_year_estimated + duration_estimated - 1 == lapse_study_year)

# Calculate split proportions for disaggregating records
df_conv_estimated <- df_conv_estimated %>% left_join(
  df_conv_estimated %>% group_by(lapse_study_year, duration, issue_year) %>%
    summarise(exposure_rate = 1/n(), .groups = "drop"),
  join_by(lapse_study_year == lapse_study_year,
    duration == duration,
    issue_year == issue_year)
) %>% arrange(lapse_study_year, duration, issue_year)

# Function to disaggregate records into single-year durations
fn_sep_to_estimated <- function(df_tmp){
  df_tmp <- df_tmp %>% left_join(df_conv_estimated,
    join_by(lapse_study_year == lapse_study_year,
      duration == duration,
      issue_year == issue_year), relationship = "many-to-many")
  for(col in c("exposure_amount", "exposure_count", "lapse_amount", "lapse_count",
    ↪ "exposure", "lapse")){
    if(col %in% names(df_tmp)){
      df_tmp <- df_tmp %>% mutate(!col := .data[[col]] * exposure_rate)
    }
  }
  df_tmp %>% select(-exposure_rate)
}

```

例えば次のレコードの場合,

```

df_tmp <- df_train_start[1:5, ]
df_tmp %>%
  select(ID, lapse_study_year, policy_year, duration,
    exposure, lapse, exposure_amount)

```

A tibble: 5 x 7

	ID	lapse_study_year	policy_year	duration	exposure	lapse	exposure_amount
	<int>	<int>	<int>	<ord>	<dbl>	<dbl>	<dbl>
1	1	2000	2000	06-09	7	1	325000
2	3	2000	2000	06-09	7	0	2600000
3	13	2001	2001	10	5	5	1800000

4	20	2000	2000 06-09	4	0	500000
5	25	2002	2002 06-09	32	3	3880000

グルーピング解除によるレコードの分割結果は次のとおりである。

```
df_tmp %>%
  fn_sep_to_estimated %>%
  select(ID, lapse_study_year, policy_year, duration, issue_year_estimated,
         duration_estimated, exposure, lapse, exposure_amount)
```

A tibble: 10 x 9

	ID	lapse_study_year	policy_year	duration	issue_year_estimated
	<int>	<int>	<int>	<ord>	<int>
1	1	2000	2000	06-09	1994
2	1	2000	2000	06-09	1995
3	3	2000	2000	06-09	1994
4	3	2000	2000	06-09	1995
5	13	2001	2001	10	1992
6	20	2000	2000	06-09	1992
7	20	2000	2000	06-09	1993
8	25	2002	2002	06-09	1994
9	25	2002	2002	06-09	1995
10	25	2002	2002	06-09	1996

	duration_estimated	exposure	lapse	exposure_amount
	<int>	<dbl>	<dbl>	<dbl>
1	7	3.5	0.5	162500
2	6	3.5	0.5	162500
3	7	3.5	0	1300000
4	6	3.5	0	1300000
5	10	5	5	1800000
6	9	2	0	250000
7	8	2	0	250000
8	9	10.667	1	1293333.
9	8	10.667	1	1293333.
10	7	10.667	1	1293333.

2.2 将来契約ポートフォリオ作成用補助関数

将来時点の契約ポートフォリオを作るための補助関数をいくつか定義しておく。次の `fn_shift_year()` 関数はデータの年限を進めるために、単純に `policy_year` と `duration_estimated` に特定の数値を加算するものである。

```
fn_shift_year <- function(df_sep, year){
  df_sep %>% mutate(policy_year = policy_year + year) %>%
```

```
mutate(duration_estimated = duration_estimated + year)
}
```

例えば、前述のレコードに対して5年シフトすると次のとおりである。

```
df_tmp %>%
  fn_sep_to_estimated %>%
  fn_shift_year(year = 5) %>%
  select(ID, lapse_study_year, policy_year, duration, issue_year_estimated,
         duration_estimated, exposure, lapse, exposure_amount)
```

A tibble: 10 x 9

	ID	lapse_study_year	policy_year	duration	issue_year_estimated
	<int>	<int>	<dbl>	<ord>	<int>
1	1	2000	2005	06-09	1994
2	1	2000	2005	06-09	1995
3	3	2000	2005	06-09	1994
4	3	2000	2005	06-09	1995
5	13	2001	2006	10	1992
6	20	2000	2005	06-09	1992
7	20	2000	2005	06-09	1993
8	25	2002	2007	06-09	1994
9	25	2002	2007	06-09	1995
10	25	2002	2007	06-09	1996

	duration_estimated	exposure	lapse	exposure_amount
	<dbl>	<dbl>	<dbl>	<dbl>
1	12	3.5	0.5	162500
2	11	3.5	0.5	162500
3	12	3.5	0	1300000
4	11	3.5	0	1300000
5	15	5	5	1800000
6	14	2	0	250000
7	13	2	0	250000
8	14	10.667	1	1293333.
9	13	10.667	1	1293333.
10	12	10.667	1	1293333.

次の `fn_get_port_eopy()` 関数は、当年度の解約実績を適用した1年後の保有契約ポートフォリオを作るための補助関数である。

```
fn_get_port_eopy <- function(df_sep){
  # If only exposure_amount is present, decrement it proportionally using exposure counts
  if ("exposure" %in% names(df_sep) & "lapse" %in% names(df_sep) &
      "exposure_amount" %in% names(df_sep) & !("lapse_amount" %in% names(df_sep))){
```

```

df_sep <- df_sep %>%
  mutate(exposure_amount = exposure_amount * (exposure - lapse) / exposure)
}
for(col in list(c("exposure_amount", "lapse_amount"),
  c("exposure_count", "lapse_count"),
  c("exposure", "lapse"))){
  if(col[1] %in% names(df_sep) & col[2] %in% names(df_sep)){
    df_sep <- df_sep %>%
      mutate(!col[1] := .data[[col[1]]] - .data[[col[2]]]) %>%
      mutate(!col[2] := 0)
  }
}
df_sep %>% fn_shift_year(year = 1)
}

```

例えば、前述のレコードに対してこの関数を適用すると次のとおりである。

```

df_tmp %>%
  fn_sep_to_estimated %>%
  fn_get_port_eopy %>%
  select(ID, lapse_study_year, policy_year, duration, issue_year_estimated,
    duration_estimated, exposure, lapse, exposure_amount)

```

A tibble: 10 x 9

	ID	lapse_study_year	policy_year	duration	issue_year_estimated
	<int>	<int>	<dbl>	<ord>	<int>
1	1	2000	2001	06-09	1994
2	1	2000	2001	06-09	1995
3	3	2000	2001	06-09	1994
4	3	2000	2001	06-09	1995
5	13	2001	2002	10	1992
6	20	2000	2001	06-09	1992
7	20	2000	2001	06-09	1993
8	25	2002	2003	06-09	1994
9	25	2002	2003	06-09	1995
10	25	2002	2003	06-09	1996

	duration_estimated	exposure	lapse	exposure_amount
	<dbl>	<dbl>	<dbl>	<dbl>
1	8	3	0	139286.
2	7	3	0	139286.
3	8	3.5	0	1300000
4	7	3.5	0	1300000
5	11	0	0	0
6	10	2	0	250000
7	9	2	0	250000

8	10	9.6667	0	1172083.
9	9	9.6667	0	1172083.
10	8	9.6667	0	1172083.

予測モデルにデータを投入する際は *_estimated ではない元のキーにその情報を反映しておく必要がある。次の fn_set_from_estimated() 関数はそのための補助関数である。

```
fn_set_from_estimated <- function(df_sep){
  df_sep %>%
    left_join(
      df_conv_duration_estimated %>% rename(duration_new = duration),
      join_by(duration_estimated == duration_estimated),
      relationship = "many-to-one"
    ) %>%
    mutate(duration = duration_new) %>%
    left_join(
      df_conv_issue_year_estimated %>% rename(issue_year_new = issue_year),
      join_by(issue_year_estimated == issue_year_estimated),
      relationship = "many-to-one"
    ) %>%
    mutate(issue_year = issue_year_new) %>%
    select(-duration_estimated, -issue_year_estimated,
           -duration_new, -issue_year_new)
}
```

前述のレコードに対してこの関数を適用すると次のとおりである。

```
df_tmp %>%
  fn_sep_to_estimated %>%
  fn_shift_year(year = 5) %>%
  fn_set_from_estimated %>%
  select(ID, lapse_study_year, policy_year, issue_year, duration, exposure, lapse)
```

A tibble: 10 x 7

	ID	lapse_study_year	policy_year	issue_year	duration	exposure	lapse
	<int>	<int>	<dbl>	<ord>	<ord>	<dbl>	<dbl>
1	1	2000	2005	1994-1996	12	3.5	0.5
2	1	2000	2005	1994-1996	11	3.5	0.5
3	3	2000	2005	1994-1996	12	3.5	0
4	3	2000	2005	1994-1996	11	3.5	0
5	13	2001	2006	1991-1993	13-	5	5
6	20	2000	2005	1991-1993	13-	2	0
7	20	2000	2005	1991-1993	13-	2	0
8	25	2002	2007	1994-1996	13-	10.7	1
9	25	2002	2007	1994-1996	13-	10.7	1
10	25	2002	2007	1994-1996	12	10.7	1

2.3 将来保険料収入の設定

元論文に記載のとおり、将来利益は各年度の将来保険料収入に定率のプロフィットマージン率を乗じたものとしてモデル化している。本データセットには保険金額にあたるデータはあるが、保険料のデータそのものはない。ART の場合を含む、保険期間の短い平準払定期保険の場合、ローディングを無視することで（年単位の）予定死亡率 $\times S$ という形で概算できるため、本研究ではそのようにモデル化している。

報告書 P7 によると、ART の場合は米国の 2001CSO という業界共通の生命表の一定率として設定されることが多いのだという。これを参考に、本研究では 2001CSO を予定死亡率の基準として採用する。

CSO は選択・終局表の形をしており、かつ男女・健康体区別に細分化されている。2001 CSO Preferred Class Structure Mortality Report & Tables から取得できるエクセルデータは扱いやすいが、一部の健康体区分の種別（喫煙区分無、喫煙区分有だが健康体区分無）はこのエクセルからは取得できないため、別途 Mortality and Other Rate Tables から手動で取得する。

保険年齢（ANB; Age Nearest Birthday）と満年齢（ALB; Age Last Birthday）のデータ双方が存在する。SOA データセットの年齢がどちらを指しているかについて特に報告書で特定されていないと思われるが、今回は米国で一般的な ANB を仮定する。

```
df_mort_dur6 <- read_excel("cso\\2001 CSO.xlsx", sheet = "Dur6", range = "A1:D1801")
df_mort_ult <- read_excel("cso\\2001 CSO.xlsx", sheet = "Ult", range = "A1:D2179")

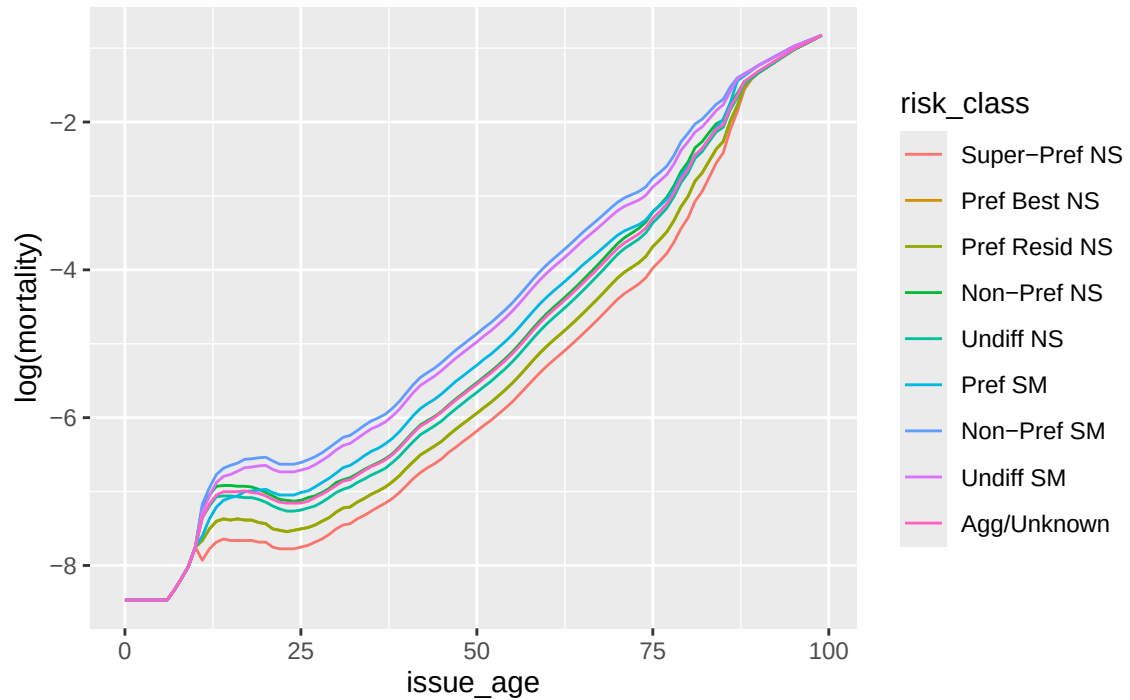
df_mort_dur6 <- df_mort_dur6 %>%
  mutate(risk_class = factor(risk_class, levels = levels(df_all$risk_class))) %>%
  mutate(gender = factor(gender, levels = c("M", "F"),
                        labels = levels(df_all$gender)))
df_mort_ult <- df_mort_ult %>%
  mutate(risk_class = factor(risk_class, levels = levels(df_all$risk_class))) %>%
  mutate(gender = factor(gender, levels = c("M", "F"),
                        labels = levels(df_all$gender)))
```

具体的にどの区分にどの生命表をあてはめるかは元論文に記載のとおり。

各年齢・性別の死亡率をグラフにすると次のとおり。ただし、縦軸は対数をとっている。

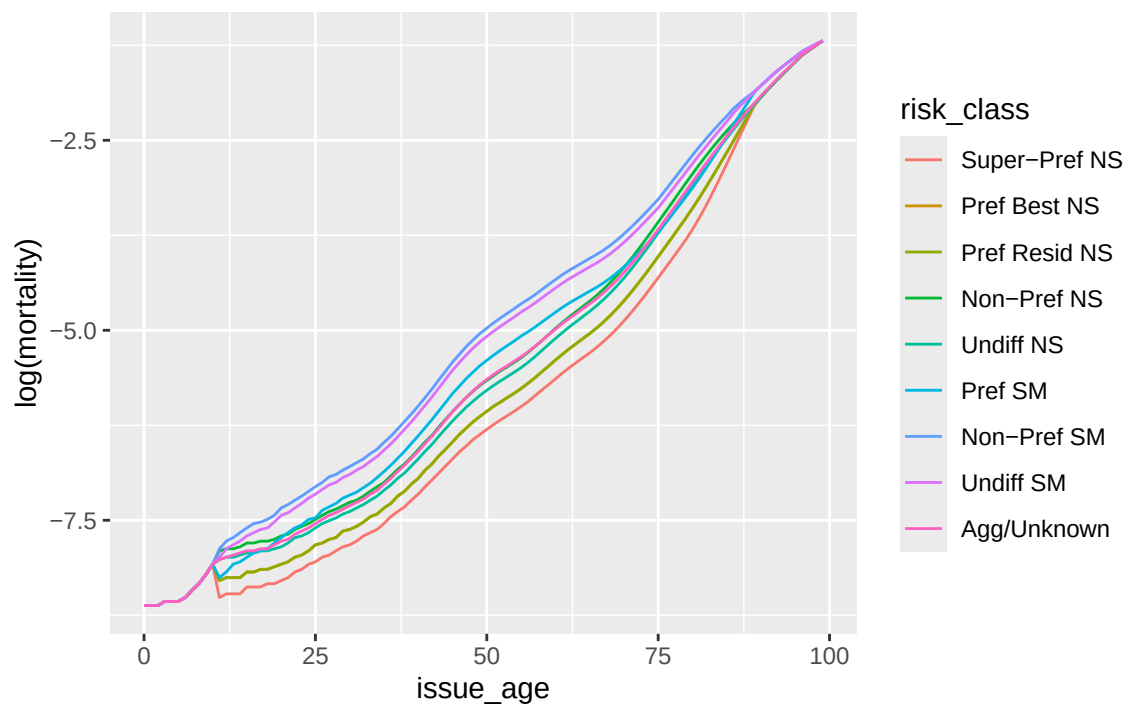
```
ggplot(df_mort_dur6 %>% filter(gender == "male"),
       mapping = aes(x = issue_age, y = log(mortality), group = risk_class, color =
                     ⇨ risk_class)) +
  geom_line() + ggtitle("2001 CSO Mortality Male Duration = 6")
```

2001 CSO Mortality Male Duration = 6

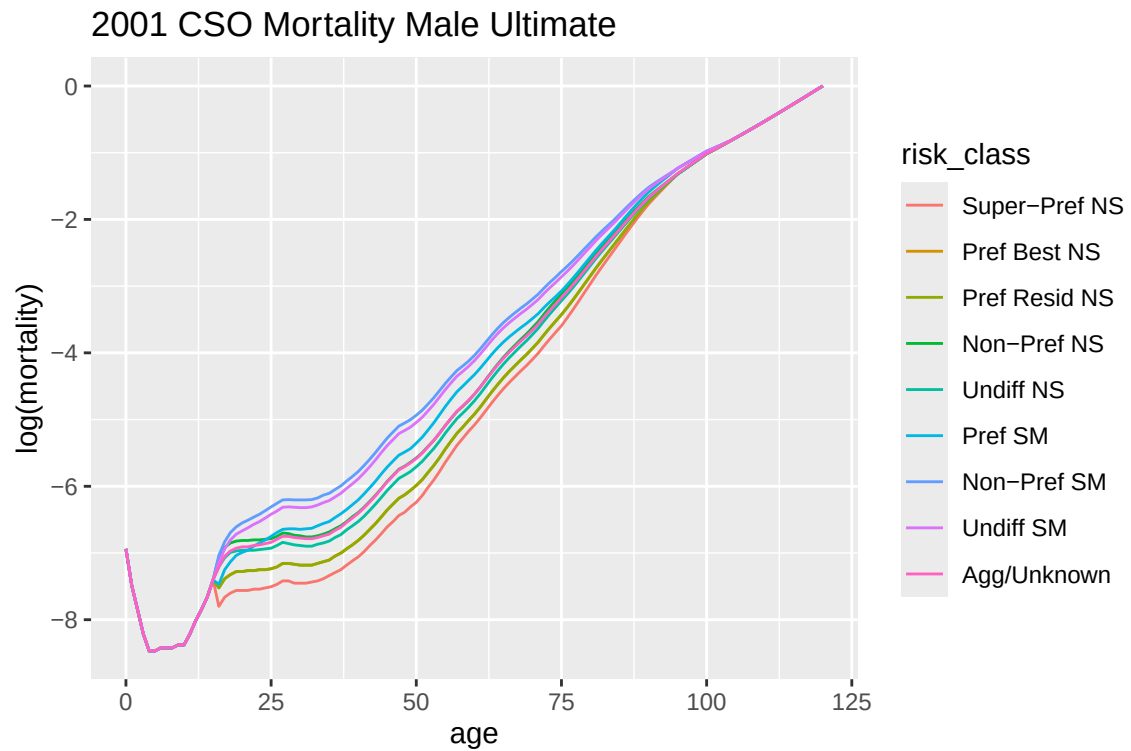


```
ggplot(df_mort_dur6 %>% filter(gender == "female"),
       mapping = aes(x = issue_age, y = log(mortality), group = risk_class, color =
         ⇐ risk_class)) +
  geom_line() + ggtitle("2001 CSO Mortality Female Duration = 6")
```

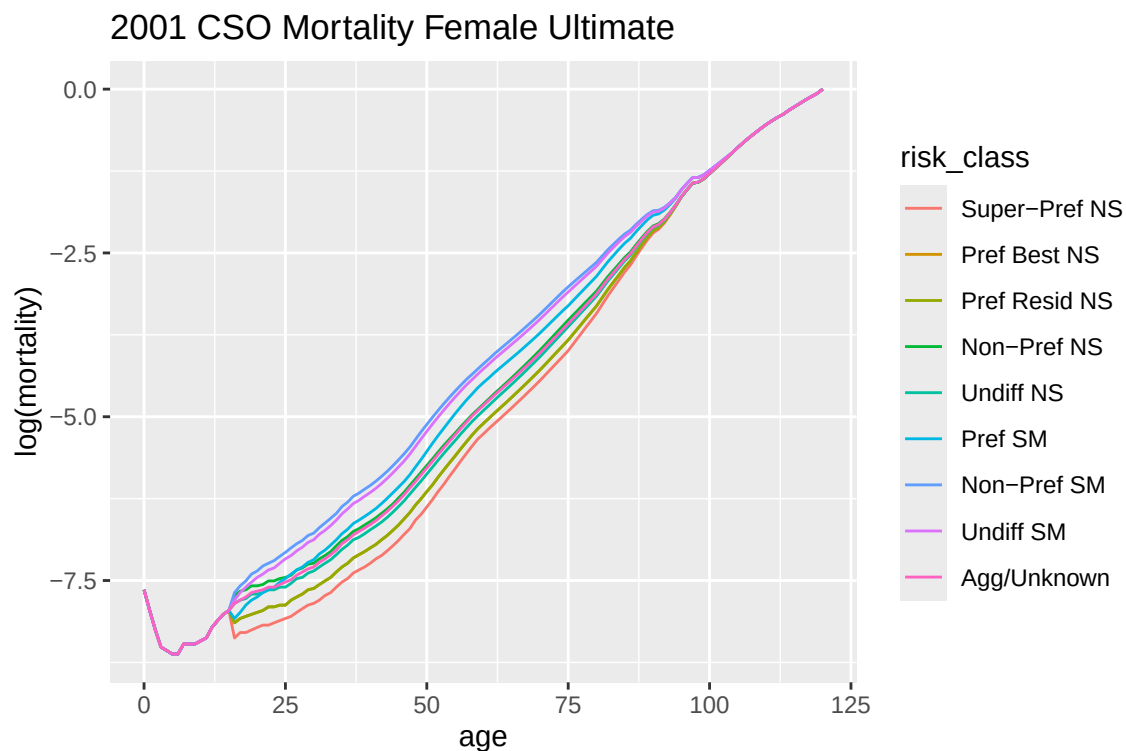
2001 CSO Mortality Female Duration = 6



```
ggplot(df_mort_ult %>% filter(gender == "male"),
       mapping = aes(x = age, y = log(mortality), group = risk_class, color = risk_class))
  ↪ +
  geom_line() + ggtitle("2001 CSO Mortality Male Ultimate")
```



```
ggplot(df_mort_ult %>% filter(gender == "female"),
       mapping = aes(x = age, y = log(mortality), group = risk_class, color = risk_class))
  ↪ +
  geom_line() + ggtitle("2001 CSO Mortality Female Ultimate")
```



大小関係を比較してみると、健康体区分無 ("Undiff **" に当てはめ) はそれぞれの Residual Standard ("Non-Pref **" に当てはめ) と近い水準である。また、喫煙区分無 ("Agg/Unknown" に当てはめ) は非喫煙者 Residual Standard ("Non-Pref NS" に当てはめ) と極めて近い水準である。Preferred Smoker ("Pref SM" に当てはめ) は概ねそれらを超える死亡率になっており、喫煙が死亡率に及ぼす影響の大きさが推測できる。

次の `fn_calc_premium()` 関数は、この生命表による予定死亡率 $\times S$ により保険料を計算する。

```
fn_calc_premium <- function(df_sep){
  df_sep %>%
    prep_toint_age() %>%
    mutate(issue_age = round(issue_age)) %>%
    mutate(age = issue_age + duration_estimated - 1) %>%
    # Level period uses Select mortality at duration 6 (joined by issue age)
    left_join(
      df_mort_dur6 %>% rename(mortality_dur6 = mortality),
      join_by(issue_age == issue_age, gender == gender, risk_class == risk_class),
      relationship = "many-to-one"
    ) %>%
    # Post-level ART period uses Ultimate mortality (varies by attained age)
    left_join(
      df_mort_ult %>% rename(mortality_ult = mortality),
      join_by(age == age, gender == gender, risk_class == risk_class),
      relationship = "many-to-one"
    ) %>%
    mutate(premium = if_else(duration_estimated <= 10, mortality_dur6, mortality_ult) *
      exposure_amount) %>%
    pull(premium)
}
```

```
}
```

ある契約で将来保険料の推移をグラフにしてみると次のとおり.保険料のジャンプ率は `premium_jump_ratio` が示す値よりも小さめになる傾向がある. この差異がなぜ現れているか, それを再現するにはどうすればよいかは推測が難しいため, 本研究では特段対処はしない. また, 平準保険料期間が終わっても ART とはならない場合 (`post_level_premium_structure == "Other"`) もデータには含まれているものの, 実際にどういうものなのかを特定するのが困難であるため, ART の場合と同じ扱いとする.

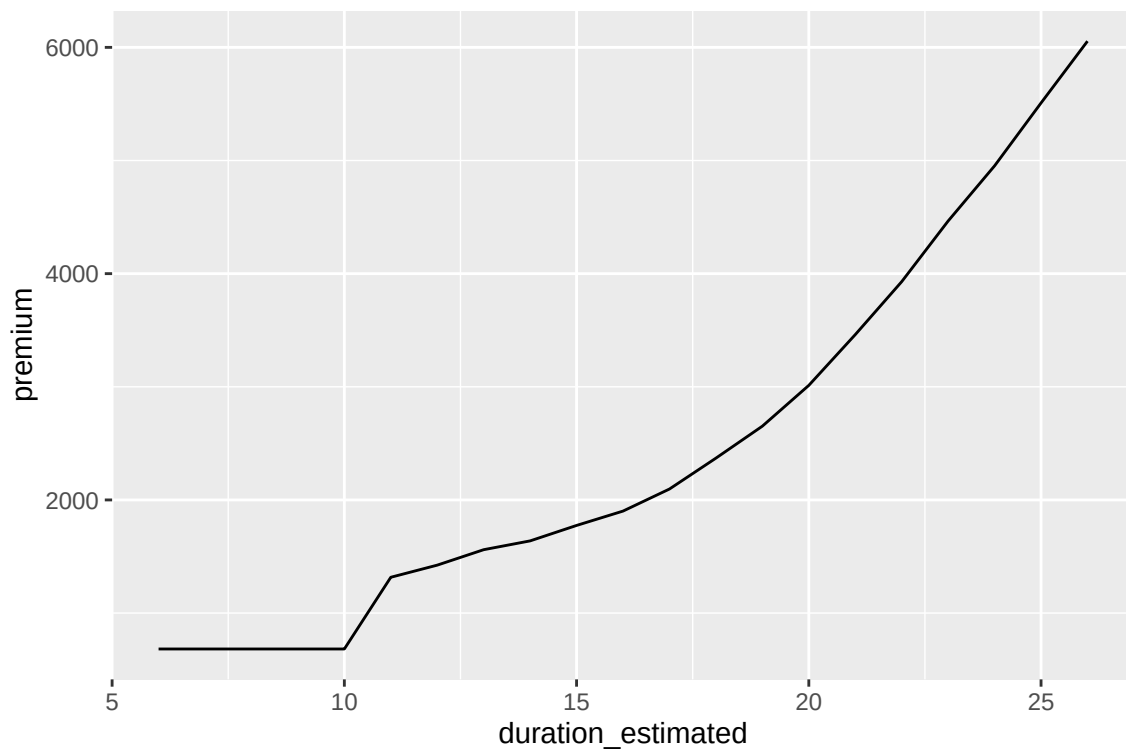
```
# Select a sample policy with a relatively large premium_jump_ratio
df_tmp <- df_test_start %>% fn_sep_to_estimated %>%
  filter(duration_estimated == 6, premium_jump_ratio == "10.01-11.00") %>%
  slice_head(n = 1)

df_tmp_all <- tibble()
for(y in 0:20){
  df_tmp_all <- rbind(df_tmp_all, df_tmp %>% fn_shift_year(year = y))
}
premium_tmp <- df_tmp_all %>% fn_calc_premium

cbind(df_tmp_all, premium = premium_tmp) %>%
  select(ID, policy_year, issue_age, duration_estimated, premium)
```

	ID	policy_year	issue_age	duration_estimated	premium
1	3066	2003	30-39	6	682.50
2	3066	2004	30-39	7	682.50
3	3066	2005	30-39	8	682.50
4	3066	2006	30-39	9	682.50
5	3066	2007	30-39	10	682.50
6	3066	2008	30-39	11	1316.25
7	3066	2009	30-39	12	1423.50
8	3066	2010	30-39	13	1560.00
9	3066	2011	30-39	14	1638.00
10	3066	2012	30-39	15	1774.50
11	3066	2013	30-39	16	1901.25
12	3066	2014	30-39	17	2096.25
13	3066	2015	30-39	18	2369.25
14	3066	2016	30-39	19	2652.00
15	3066	2017	30-39	20	3012.75
16	3066	2018	30-39	21	3461.25
17	3066	2019	30-39	22	3929.25
18	3066	2020	30-39	23	4465.50
19	3066	2021	30-39	24	4953.00
20	3066	2022	30-39	25	5508.75
21	3066	2023	30-39	26	6054.75

```
ggplot(cbind(df_tmp_all, premium = premium_tmp),
       mapping = aes(x = duration_estimated, y = premium)) +
  geom_line()
```



2.4 将来 CF 展開・利益現価計算

元論文に記載の前提のもと、将来利益現価の計算を行う関数を作成する。割引現在価値の計算の際には経過の末端部分から遡るように計算すると効率的であるため、そのように実装している。

```
# projection_term : Projection Horizon
# verbose :
#   - If >= 1, returns projected cash flows in addition to EV
#   - If 1, returns data for t = 0 only; if 2, returns all projection periods
# int_rate : Valuation Interest Rate
# pm_rate : Profit Margin Ratio (After Tax)
fn_calc_ev <- function(df_sep, model, predfun,
                      projection_term = 20, verbose = 0, int_rate = 0.02,
                      pm_rate = 0.05){

  if (verbose >= 1){
    df_projected_data <- tibble()
  }
  for (t in (projection_term - 1):0){

    df_sep_t <- df_sep %>% fn_shift_year(year = t)
    if(t == projection_term - 1){
```

```

    ev <- rep(0, nrow(df_sep))
    if (verbose >= 1){
      lapse_rate <- ev
    }
  }else{
    lapse_rate <- predfun(model, df_sep_t %>% fn_set_from_estimated)
    ev <- (1 - lapse_rate) / (1 + int_rate) * ev
    if (verbose >= 1){
      df_projeted_data$cf <- df_projeted_data$cf * (1 - lapse_rate)
    }
  }
}
premium <- df_sep_t %>% fn_calc_premium
if (verbose >= 1){
  df_projeted_data <- rbind(
    data.frame(
      rowid = 1:nrow(df_sep),
      t = rep(t, nrow(df_sep)),
      premium = premium,
      lapse_rate = lapse_rate,
      cf = premium
    ),
    df_projeted_data
  )
}

ev <- ev + premium * pm_rate
}
if (verbose >= 1){
  if (verbose <= 1){
    df_projeted_data <- df_projeted_data %>% filter(t == 0)
  }
  return(list(ev = ev, df_projeted_data = df_projeted_data))
}else{
  return(list(ev = ev))
}
}

```

前の節で抽出した1件のサンプルで計算した場合の各年度の保険料キャッシュフローや解約率等の計算結果は次のとおりである。duration == 10での解約率上昇を反映し、以降の期間の保険料キャッシュフローが非常に小さくなっていることがわかる。

```

ret <- fn_calc_ev(
  df_tmp,
  model_xgboost_start, predfun_xgboost,
  verbose = 2)
ret$df_projeted_data

```

	rowid	t	premium	lapse_rate	cf
1	1	0	682.50	0.07321494	682.50000
119	1	1	682.50	0.07321494	632.53080

118	1	2	682.50	0.07321494	586.22010
117	1	3	682.50	0.07321494	543.30003
116	1	4	682.50	0.72974104	503.52235
115	1	5	1316.25	0.46208256	262.44275
114	1	6	1423.50	0.23819400	152.67548
113	1	7	1560.00	0.16113362	127.46202
112	1	8	1638.00	0.16113362	112.26979
111	1	9	1774.50	0.16113362	102.02763
110	1	10	1901.25	0.16113362	91.70094
19	1	11	2096.25	0.16113362	84.81457
18	1	12	2369.25	0.16113362	80.41389
17	1	13	2652.00	0.16113362	75.50687
16	1	14	3012.75	0.16113362	71.95630
15	1	15	3461.25	0.16113362	69.34761
14	1	16	3929.25	0.16113362	66.03908
13	1	17	4465.50	0.16113362	62.95848
12	1	18	4953.00	0.16113362	58.57944
11	1	19	5508.75	0.00000000	54.65410

この保険料キャッシュフローに対してプロフィットマージン率を乗じ、割引現在価値をとったものが EV である。

```
ret$ev[[1]]
```

```
[1] 202.5765
```

最後に、テストデータ全体に対して EV を計算すると次のとおり。

```
df_test_start_sep <- df_test_start %>% fn_sep_to_estimated

ev_result_start <- fn_calc_ev(
  df_test_start_sep,
  model_xgboost_start, predfun_xgboost,
  verbose = 1) # Used later to construct the expected portfolio reflecting projected
               ↪ lapses.
sum(ev_result_start$ev)
```

3 EV 変動要因分析の計算

元論文に記載の方法で変動要因分析を行う。ここでは評価年度 (Evaluation Year) 2002 年度末から 2003 年度末への EV 変動要因分析を題材とする。2002 年度末、すなわち期初時点の EV は前の節で計算した `ev_result_start` である。これを出発点に、2003 年度末までの変動要因分析を作成する。

まず、テストデータ全体を期初の保有契約ポートフォリオとみなし、期初時点の解約率異動予測を反映した保有契約ポートフォリオ、実際の解約異動を反映した保有契約ポートフォリオをそれぞれ計算する。

```
df_test_start_end_pred <- df_test_start_sep %>%
  cbind(pred = ev_result_start$df_projected_data %>%
    filter(t == 0) %>% select(lapse_rate) %>% pull) %>%
  mutate(exposure = exposure * (1 - pred)) %>%
  mutate(exposure_amount = exposure_amount * (1 - pred)) %>%
  select(-pred) %>% fn_shift_year(year = 1)

df_test_start_end_act <- df_test_start_sep %>% fn_get_port_eopy
```

これで各シナリオの解約率モデルと保有契約ポートフォリオのデータが揃ったため、順次それらを計算していく。時点異動を反映するためのシナリオ A を計算する。保有契約ポートフォリオを期初時点の解約率異動予測を反映したものに差し替える。

```
ev_result_a <- fn_calc_ev(
  df_test_start_end_pred,
  model_xgboost_start, predfun_xgboost,
  verbose = 0)
sum(ev_result_a$ev)
```

[1] 63730019

次に、前提実績差異を計算するためのシナリオ B を計算する。保有契約ポートフォリオを実際の解約異動に反映したものに差し替える。

```
ev_result_b <- fn_calc_ev(
  df_test_start_end_act,
  model_xgboost_start, predfun_xgboost,
  verbose = 0)
sum(ev_result_b$ev)
```

[1] 64431748

さらに、前提変更差異を計算するためのシナリオ C を計算する。解約率予測モデルを期末時点のものに変更する。

```
ev_result_c <- fn_calc_ev(
  df_test_start_end_act,
  model_xgboost_end, predfun_xgboost,
  verbose = 0)
sum(ev_result_c$ev)
```

[1] 71543997

最後に、保有契約ポートフォリオを期末時点のテストデータに変更することで、次年度の EV が計算され、変動要因分析に必要なすべての計算が揃う。

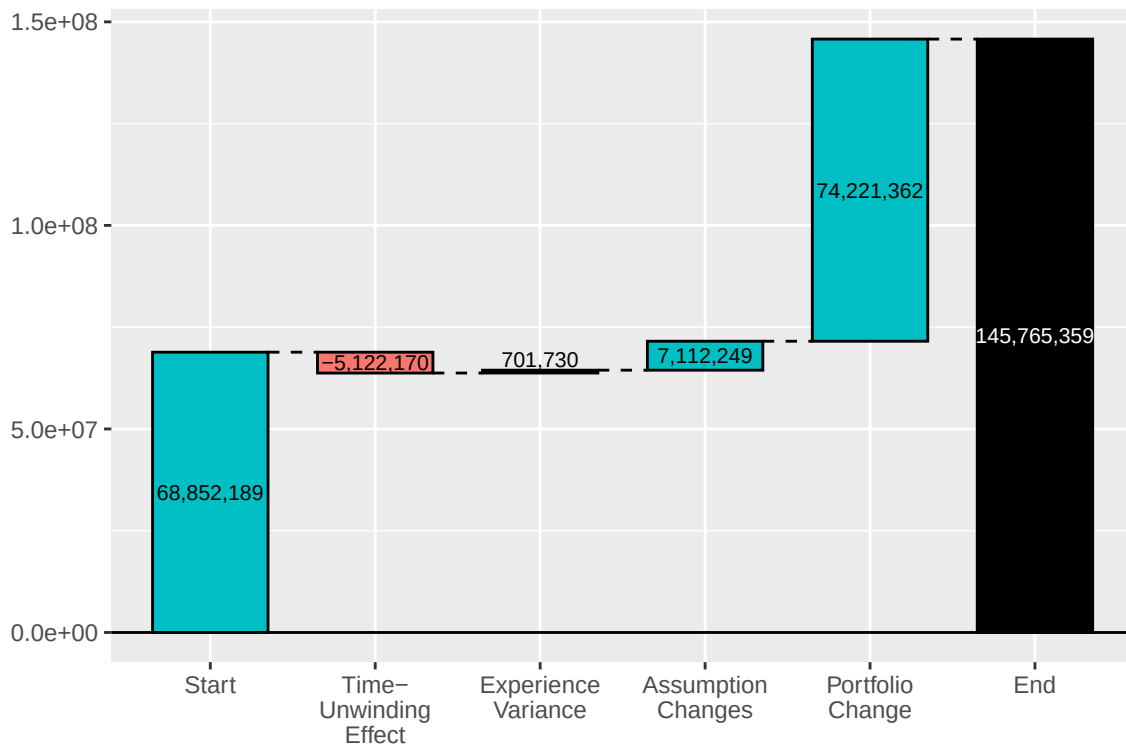
```
ev_result_end <- fn_calc_ev(
  df_test_end %>% fn_sep_to_estimated,
  model_xgboost_end, predfun_xgboost,
  verbose = 0)
sum(ev_result_end$ev)
```

[1] 145765359

これらの計算結果に基づいて変動を計算し、ウォーターフォール図にすると次のとおり。元論文の結果が再現されている。

```
vec_values <- c(sum(ev_result_start$ev)
, sum(ev_result_a$ev) - sum(ev_result_start$ev)
, sum(ev_result_b$ev) - sum(ev_result_a$ev)
, sum(ev_result_c$ev) - sum(ev_result_b$ev)
, sum(ev_result_end$ev) - sum(ev_result_c$ev))

waterfall(
  labels = c(
    "Start", "Time-\nUnwinding\nEffect", "Experience\nVariance",
    "Assumption\nChanges", "Portfolio\nChange"),
  total_axis_text = "End",
  values = vec_values,
  rect_text_labels = scales::comma(vec_values, accuracy = 1),
  total_rect_text = scales::comma(sum(vec_values), accuracy = 1),
  calc_total = T
)
```



Acknowledgments

本研究の実施にあたり、公表データ (SOA 10-Year Level Premium Term Lapse Study) を提供いただいた Society of Actuaries (SOA) に深謝いたします。なお、本論文の作成プロセスにおける英文表現の推敲およびコード構築の補助にあたっては、適切な人間による確認と責任のもとで生成 AI ツール (Gemini / ChatGPT

等)を活用いたしました。

References

Kozuma, R., Asashiba, R., Matsumori, Y., & Iwasawa, H. (2026). *Enhancing GLM-based lapse rate models for EV calculation using interpretable machine learning.*